

AIM2- Preparations

DAY 6 — Files, Exceptions & Modules

Now we're going to learn something very important:

Until now, your programs have mostly been:

```
Program starts
  ↓
User enters data
  ↓
Program processes it
  ↓
Program ends
```

But real applications need to **save and load data**.

That's where files come in.

Opening a file

Python:

```
file = open("students.txt", "r")
```

Modes:

"r" → read

"w" → write

"a" → append

But there's a better way.

Use:

```
with open("students.txt", "r") as file:  
    data = file.read()
```

The **with statement** automatically handles closing the file.

2 Writing a file

```
with open("students.txt", "w") as file:  
    file.write("Krishna,82.67\n")  
    file.write("Rahul,83.67\n")
```

Now Python creates/writes:

students.txt

3 Reading a file

```
with open("students.txt", "r") as file:  
    data = file.read()  
  
print(data)
```

4 Reading line by line

```
with open("students.txt", "r") as file:  
  
    for line in file:  
        print(line.strip())
```

This is another place where your for loop knowledge becomes useful.

5 Exceptions

What happens if the user enters:

abc

when you're expecting:

```
age = int(input("Age: "))
```

Python throws an error.

We can handle it:

```
try:
    age = int(input("Enter your age: "))
except ValueError:
    print("Please enter a valid number.")
```

Concept:

```
try
↓
Try dangerous/uncertain operation

except
↓
Handle the error
```

6 finally

You can also have:

```
try:
    number = int(input("Enter number: "))

except ValueError:
    print("Invalid number.")

finally:
    print("Program finished.")
```

finally runs whether there is an error or not.

7 Modules

You've already used modules indirectly.

For example:

```
import math

Then:

print(math.sqrt(25))
```

Output:

5.0

You can also import specific things:

```
from math import sqrt
```

Then:

```
print(sqrt(25))
```

8 Creating your own module

Suppose you create:

```
calculator.py
```

Inside:

```
def add(a, b):  
    return a + b
```

Then another Python file can do:

```
import calculator  
  
print(calculator.add(10, 20))
```

This is the foundation of organizing larger Python applications.

9 Classes — Python OOP connection to Java

You already know Java OOP, so this should feel familiar.

Java:

```
class Student {  
    String name;  
    int age;  
}
```

Python:

```
class Student:  
  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

Create an object:

```
student = Student("Krishna", 28)
```

Access:

```
print(student.name)  
print(student.age)
```

Output:

```
Krishna  
28
```

Java → Python connection

Java	Python
class	class
constructor	<code>__init__()</code>
this	self
method	method
object	object
inheritance	inheritance

Your Java background is going to help you **a lot** here.



DAY 6 PRACTICE

Create:

`day-06-files-exceptions-modules.py`

Exercise 1 — File Writing

Create a file:

`students.txt`

Write:

Krishna,82.67

Rahul,83.67

Priya,91.00

Exercise 2 — File Reading

Read the file and print each student.

Exercise 3 — Exception Handling

Ask:

Enter your age:

Handle invalid input.

For example:

Enter your age: abc

Invalid age. Please enter a number.

Your program should **not crash**.

Exercise 4 — Create Your Own Module

Create:

`student_utils.py`

Put functions inside:

```
calculate_total()
calculate_average()
calculate_grade()
```

Then import them into:

`day-06-files-exceptions-modules.py`

This is a very good exercise for you because you're learning how to turn your Day 4 functions into a reusable module.

Day 6 Mini Project — Student File Analyzer

The question

You are building a Python program that reads student marks from a text file and generates a performance report.

You have a file called:

`students.txt`

It contains student information in this format:

```
Name,Math,Python,AI
Krishna,95,88,65
```

Kuntal,90,92,87
Parth,98,95,94
Rahul,82,91,78
Priya,90,94,89

Your task

Create a Python program that:

1. Opens and reads `students.txt`.
2. Reads each student's name and three subject scores.
3. Converts the scores from strings to numbers.
4. Calculates:
 - a. Total score
 - b. Average score
 - c. Grade
 - d. Pass/Fail result
5. Displays each student's result.
6. Uses your `student_utils.py` module for the calculations.
7. Uses `try/except` so the program doesn't crash if the file is missing or contains invalid data.

Expected output

Something similar to:

```
=====
      STUDENT FILE ANALYZER
=====
```

```
Student: Krishna
Math: 95
Python: 88
AI: 65
Total: 248.00
Average: 82.67
Grade: B
```

Result: PASSED

Student: Kuntal

Math: 90

Python: 92

AI: 87

Total: 269.00

Average: 89.67

Grade: B

Result: PASSED

...

Project structure

For this project, I want you to have:

```
week-01-python/  
├──  
│   └── exercises/  
│       ├── day-06-files-exceptions-modules.py  
│       ├── student_utils.py  
│       └── students.txt
```

You already have `student_utils.py`, so we'll reuse it.

Step 1 — `students.txt`

Put this inside:

```
Name,Math,Python,AI
Krishna,95,88,65
Kuntal,90,92,87
Parth,98,95,94
Rahul,82,91,78
Priya,90,94,89
```

Notice the first line:

```
Name,Math,Python,AI
```

That's a **header**. We don't want to process it as a student.

Step 2 — student_utils.py

You already created this file, but let's make the functions consistent with the project.

```
def calculate_total(math, python, ai):
    return math + python + ai
```

```
def calculate_average(total):
    return total / 3
```

```
def calculate_grade(average):
    if average >= 90:
        return "A"
    elif average >= 80:
        return "B"
    elif average >= 70:
        return "C"
    elif average >= 60:
        return "D"
    else:
        return "F"
```

```
def calculate_result(grade):  
    if grade == "F":  
        return "FAILED"  
    else:  
        return "PASSED"
```

Notice something important

We're keeping your design from your Exercise 4:

```
calculate_total()  
    ↓  
calculate_average()  
    ↓  
calculate_grade()  
    ↓  
calculate_result()
```

So you're not learning a completely new design.

Step 3 — Main program

Now comes the actual mini-project.

Create/open:

`day-06-student-file-analyzer.py`

and start with:

```
import student_utils
```

We're importing your own module.

Step 4 — Open the file

```
try:
    with open("students.txt", "r") as file:
        lines = file.readlines()

except FileNotFoundError:
    print("Error: students.txt was not found.")
```

What's happening?

`readlines()` gives us a list.

For example:

```
[
    "Name,Math,Python,AI\n",
    "Krishna,95,88,65\n",
    "Kuntal,90,92,87\n"
]
```

Step 5 — Skip the header

We don't want:

Name,Math,Python,AI

So:

```
for line in lines[1:]:
```

The `[1:]` means:

Start from the second line.

Step 6 — Remove the newline

Each line contains `\n`.

So:

```
line = line.strip()
```

For example:

```
"Krishna,95,88,65\n"
```

becomes:

```
"Krishna,95,88,65"
```

Step 7 — Split the data

Now:

```
data = line.split(",")
```

This converts:

```
Krishna,95,88,65
```

into:

```
["Krishna", "95", "88", "65"]
```

This is a very important Python concept.

Step 8 — Extract the values

We can use:

```
name = data[0]
math = float(data[1])
python = float(data[2])
ai = float(data[3])
```

Why float()?

Because data read from a file is initially **text/string**.

So:

"95"

is a string.

But:

95.0

is a number.

We need numbers to perform calculations.

Step 9 — Use your module

Now:

```
total = student_utils.calculate_total(math, python, ai)
average = student_utils.calculate_average(total)
grade = student_utils.calculate_grade(average)
result = student_utils.calculate_result(grade)
```

This is the part I really want you to understand.

Your main program doesn't need to know **how** the calculations work.

It simply says:

```
student_utils,  
calculate the total.
```

This is why modules are useful.

Step 10 — Display the result

```
print(f"Student: {name}")  
print(f"Math: {math:.2f}")  
print(f"Python: {python:.2f}")  
print(f"AI: {ai:.2f}")  
print(f"Total: {total:.2f}")  
print(f"Average: {average:.2f}")  
print(f"Grade: {grade}")  
print(f"Result: {result}")  
print("-----")
```

Complete Solution

Now here is the complete solution so you can compare your attempt with it.

```
import student_utils  
  
print("=====")  
print("      STUDENT FILE ANALYZER")  
print("=====")  
print()  
  
try:  
    with open("students.txt", "r") as file:  
        lines = file.readlines()  
  
    # Skip the header  
    for line in lines[1:]:
```

```

try:
    # Remove whitespace and newline
    line = line.strip()

    # Split the line into separate values
    data = line.split(",")

    # Extract student information
    name = data[0]
    math = float(data[1])
    python = float(data[2])
    ai = float(data[3])

    # Calculate student performance
    total = student_utils.calculate_total(math, python, ai)
    average = student_utils.calculate_average(total)
    grade = student_utils.calculate_grade(average)
    result = student_utils.calculate_result(grade)

    # Display result
    print(f"Student: {name}")
    print(f"Math: {math:.2f}")
    print(f"Python: {python:.2f}")
    print(f"AI: {ai:.2f}")
    print(f"Total: {total:.2f}")
    print(f"Average: {average:.2f}")
    print(f"Grade: {grade}")
    print(f"Result: {result}")
    print("-----")

except (ValueError, IndexError):
    print("Error: Invalid student data.")
    print("-----")

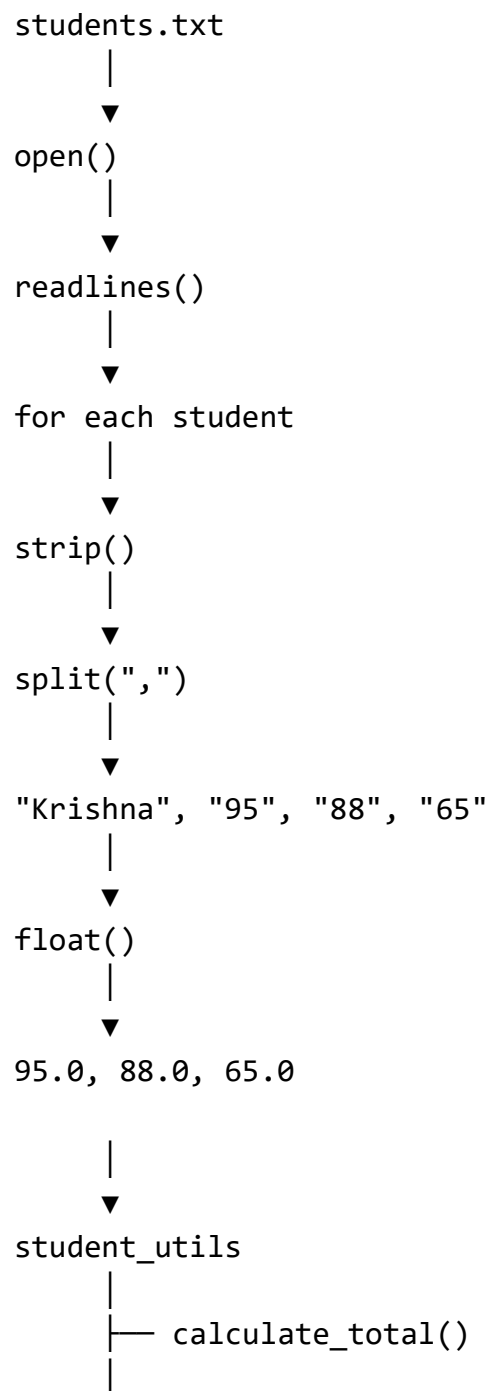
except FileNotFoundError:
    print("Error: students.txt was not found.")

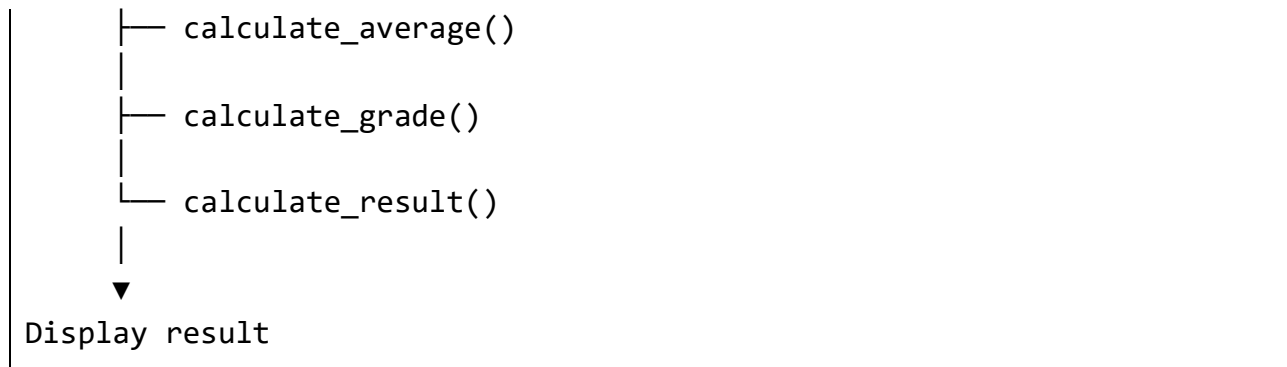
```

Let's understand the whole program

This is the important part. Don't just copy the solution.

The program flow is:





This is exactly why I wanted this project for Day 6.

You are combining **everything you've learned so far**.